Adaptive Control: the Self-Tuning PI Controller

1.0 Learning Outcomes.....1
2.0 Introduction.....1
3.0 Fixed Controllers and Changing Systems.....2
4.0 Approaches to Adaptive Control.....3
5.0 The Self-Tuning PI Controller.....3
6.0 Safeguards for Reliable Adaptation.....5
7.0 Example: Self-Tuning PI Speed Control (Simulation).....5
8.0 Summary.....7
References.....8
Recommended Citation.....8

1.0 Learning Outcomes

After studying this ACE-Lab learning guide, you should be able to demonstrate the Learning Outcomes listed in Table 1. On completion of the ACE-Lab learning guide, revisit Table 1 and confirm whether each Learning Outcome has been achieved.

Table 1: *Learning Outcomes from the Adaptive Control: the Self-Tuning PI Controller Learning Guide*

ID	Learning Outcome	Achieved
1.	Explain why a fixed-gain controller cannot maintain its designed performance when the gain or dynamics of the system change.	☐
2.	Compare the main approaches to adaptive control, including gain scheduling, model reference adaptive control and self-tuning regulators.	☐
3.	Describe the structure of an indirect self-tuning regulator that combines recursive least squares estimation with pole-placement design.	☐
4.	Identify the safeguards needed for reliable adaptive control on real hardware.	☐
5.	Interpret simulated results from a self-tuning PI speed controller, including how the estimated model and the gains respond to a change in the system.	☐

2.0 Introduction

The PI Control Design by Pole Placement Learning Guide calculated the gains of a PI controller from an identified model, so that the closed loop has a chosen damping ratio and natural frequency. The design is only as good as the model, and the model describes the system at the moment it was identified. If the system changes, for example because a load is added to a motor, the gains are no longer correct and the closed-loop response changes. An important question therefore arises:

How can a controller re-tune itself automatically when the system it controls changes?

This is the aim of adaptive control. An adaptive controller adjusts its own parameters during operation, in response to measured changes in the behaviour of the system. This Learning Guide introduces the main approaches, and then develops one of the most widely used: the self-tuning regulator, which combines the Recursive Least Squares for Online Parameter Estimation and PI Control Design by Pole Placement Learning Guides. At every estimation step, the controller identifies the motor using recursive least squares (RLS) and redesigns its PI gains by pole placement.

Why is Adaptive Control Important?

Adaptive control is important in control engineering because:

i. Real systems change during operation: loads, friction, temperature, wear and supply voltage all alter the gain and dynamics of a system.

ii. A fixed controller tuned for one condition may become sluggish, oscillatory or even unstable when conditions change.

iii. An adaptive controller can maintain the designed response across a range of conditions without manual re-tuning.

iv. The online model is a useful by-product: it can be used to monitor the condition of a system and to detect faults.

3.0 Fixed Controllers and Changing Systems

The Effect of a Change in Gain

Consider a PI controller, $C(s) = K_p + K_i/s$, designed for a motor $G(s)$. Suppose the effective gain of the motor changes by a factor α, for example $\alpha = 0.6$ when a load is added or $\alpha = 2$ when a load is removed. The loop transfer function becomes:

$$C(s)\,\alpha G(s) = \left(\alpha K_p + \frac{\alpha K_i}{s}\right) G(s) \tag{1}$$

A change in the gain of the motor is therefore exactly equivalent to multiplying both controller gains by α. Since the design of Equation (10) in the PI Control Design by Pole Placement Learning Guide gives gains that are inversely proportional to the motor gain, the ideal response would be to divide both gains by α. A fixed controller cannot do this, so its closed-loop poles move away from the designed locations.

Example 1: Closed-loop poles of a fixed PI controller after the motor changes

A PI controller was designed for $K = 16$ RPM/V and $\tau = 0.054$ s, with $\zeta = 1$ and $\omega_n = 30$ rad/s, giving $K_p = 0.104$ V/RPM and $K_i = 2.48$ V/(RPM s) with $T_s = 0.01$ s. Find the closed-loop poles when the gain of the motor falls to 60%.

Solution:

From Equation (7) of the PI Control Design by Pole Placement Learning Guide, the characteristic equation is $z^2 + (b_s q_0 - 1 - p)z + (p + b_s q_1) = 0$, with $p = 0.8310$, $q_0 = K_p + K_i T_s = 0.1291$ and $q_1 = -K_p = -0.1043$. With the reduced gain, $b_s = 0.6 \times 16(1-0.8310) = 1.623$:

$$z^2 + (1.623 \times 0.1291 - 1.8310)z + (0.8310 - 1.623 \times 0.1043) = 0$$

$$z^2 - 1.6214z + 0.6617 = 0$$

The poles are $z = 0.811 \pm j0.067$. Mapping back with $s = \ln(z)/T_s$ gives $\omega_n \approx 22$ rad/s and $\zeta \approx 0.93$: the loop is now about 25% slower than designed. When the gain doubles instead, the poles split to $z = 0.798$ and $z = 0.334$, and the response is no longer the designed critically damped one. Redesigning with the new gain, by Equation (10), gives $K_p = 0.174$ V/RPM and $K_i = 4.14$ V/(RPM s) for the weaker motor, and restores the poles to the double pole at $z = 0.741$.

4.0 Approaches to Adaptive Control

There are several ways to make a controller adapt, summarised in Table 2 (Åström and Wittenmark, 1995).

Table 2: *Main Approaches to Adaptive Control*

Approach	How the controller adapts	Strengths	Limitations
Gain scheduling	Gains are switched or interpolated from a table, according to a measured operating condition (for example speed or load).	Simple, predictable and widely used in industry.	The schedule must be designed in advance; unforeseen changes are not handled.
Model reference adaptive control (MRAC)	The gains are adjusted so that the closed loop follows a reference model, using an adaptation law driven by the tracking error.	Direct adaptation; stability can be proven for some systems.	Adaptation laws can be difficult to tune and to analyse.
Self-tuning regulator (STR)	A model is estimated online, and the controller is redesigned from the latest model at every step.	Transparent: the model and the design rule can both be inspected.	Needs excitation and safeguards; relies on a suitable model structure.
Learning-based tuning	A policy, learned from experience (for example by reinforcement learning), chooses the gains.	Can learn complex strategies without an explicit model.	Needs a good simulator and training; decisions are hard to explain.

This Learning Guide focuses on the self-tuning regulator. Learning-based tuning is the subject of the Reinforcement Learning for PI Controller Tuning Learning Guide.

5.0 The Self-Tuning PI Controller

Structure

The self-tuning PI controller is an indirect adaptive controller: it first estimates a model of the system, and then calculates the controller from the model. It contains two loops that run at different rates, as shown in Figure 1. The inner, feedback loop is an ordinary PI speed controller, running every $T_s = 0.01$ s. The outer,

adaptation loop runs every T_e = 0.05 s: an RLS estimator updates the model of the motor from the applied voltage and the measured speed, and a design block recalculates the PI gains from the latest model by pole placement.

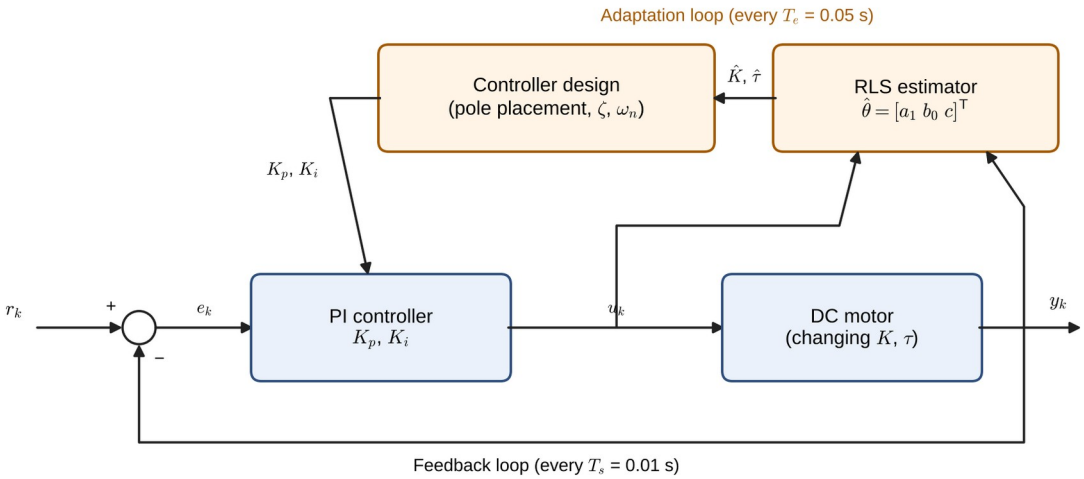


Figure 1: *Structure of the Self-Tuning PI Controller: an RLS Estimator and a Pole-Placement Design Adjust the Gains of the PI Feedback Loop*

The design uses the latest estimate as if it were the true model. This is known as the certainty equivalence principle. It is simple and effective when the estimate is good, but it means that a poor estimate produces poor gains, which is why the safeguards in Section 6.0 are essential.

The Algorithm

At every estimation step, the self-tuning PI controller carries out the following steps.

Step 1: Update the model. The RLS estimator, with forgetting factor λ, updates the parameters of the first-order model with an offset, $\hat{\theta} = [a_1\ \ b_0\ \ c]^T$:

$$y_k = -a_1 y_{k-1} + b_0 u_{k-1} + c \tag{2}$$

Step 2: Convert the model to the control interval. The gain and the discrete-time pole at the control interval $T_s = T_e/D$ are:

$$\hat{K} = \frac{b_0}{1 + a_1}, \qquad p = (-a_1)^{1/D}, \qquad b_s = \hat{K}(1 - p) \tag{3}$$

Step 3: Redesign the PI gains by pole placement. With the required poles p_1 and p_2 from ζ and ω_n:

$$K_p = \frac{p - p_2}{b_s}, \qquad K_i = \frac{1 + p_1 + p_2}{b_s T_s} \tag{4}$$

Step 4: Apply the control law. At every control sample, the velocity-form PI law uses the latest gains, with the output clamped to the actuator limits:

$$u_k = u_{k-1} + (K_p + K_i T_s)\, e_k - K_p e_{k-1}, \qquad 0 \le u_k \le V_s \tag{5}$$

The velocity form is important here: because it calculates a change in the control signal rather than its absolute value, the output does not jump when the gains change, so the gains can be updated at every step without disturbing the motor.

6.0 Safeguards for Reliable Adaptation

An adaptive controller is a nonlinear, time-varying system, and the estimator and controller interact: the controller affects the data from which the model is estimated. Without safeguards, a poor estimate can produce unsuitable gains, which produce poor data, which produce a worse estimate. Table 3 lists the safeguards used in the ACE-Lab self-tuning PI controller.

Table 3: *Safeguards in the ACE-Lab Self-Tuning PI Controller*

Safeguard	Implementation	Purpose
Initial guess	RLS starts from K_0 = 10 RPM/V and τ_0 = 0.3 s, with $P_0 = 100I$; fixed gains K_p = 0.1, K_i = 0.5 are used until the model is valid.	Safe behaviour while the estimate settles
Validity check	Gains are only updated when $0 < p < 1$ and $\hat{K} > 0$; otherwise the previous gains are kept.	Rejects physically impossible models
Gain limits	$0 \le K_p \le 1$ V/RPM and $0 \le K_i \le 10$ V/(RPM s).	Prevents extreme gains from a poor estimate
Anti-windup of the estimator	λ is set to 1 whenever trace(P) > 10^4.	Prevents covariance wind-up
Anti-windup of the controller	u_k is clamped to the actuator limits.	Prevents integrator windup
Offset term	The constant c in Equation (2) models the dead-band.	Removes bias in the estimated gain
Matched filtering	The voltage passes through the same low-pass filter as the speed before estimation.	Removes the filter lag from the estimated time constant
Excitation	The speed reference changes every 3 s.	Keeps the data informative

The choice of forgetting factor and design target follows the earlier Learning Guides. A memory of about 5 s (λ = 0.99 with T_e = 0.05 s) allows the controller to follow a sudden change in load within 10 to 20 s, while keeping the gains steady. The natural frequency should be chosen between $1/\tau$ and $5/\tau$ for the expected range of motors: a self-tuning controller recalculates the gains, but it cannot correct a design target that is too slow or too fast.

7.0 Example: Self-Tuning PI Speed Control (Simulation)

In this example, the self-tuning PI controller is applied to a virtual DC motor (`dc_motor_str_virtual.slx`) with K = 16 RPM/V, τ = 0.25 s and a 0.5 V dead-band, running at 100 Hz. The reference alternates between 20 and 40 RPM every 3 s. At t = 40 s the gain of the motor falls to 60%, to represent an added load. The controller uses ζ = 1, ω_n = 6 rad/s (for this slower virtual motor, $2\zeta\omega_n\tau = 3$, so the target is realistic) and λ = 0.99. The core of the controller is the MATLAB Function block below.

MATLAB Function: self-tuning PI controller (pole placement from the latest RLS model)

```
function [u, Kp, Ki] = self_tuning_pi(r, y, theta, Ts, D, zeta, wn, ...
                                      Vs, adapt_on, Kp0, Ki0, Kp_max, Ki_max)
```

```
persistent uPrev ePrev KpS KiS
if isempty(uPrev)
    uPrev = 0;  ePrev = 0;  KpS = Kp0;  KiS = Ki0;   % safe initial gains
end
%% 1-3. Recalculate the PI gains from the latest model
if adapt_on > 0
    a1 = theta(1);  b0 = theta(2);
    pe = -a1;                                   % pole at the estimation rate
    if pe > 0 && pe < 1 && abs(1 + a1) > 1e-9
        K = b0/(1 + a1);                        % gain [RPM/V]
        p = pe^(1/D);                           % pole at the control rate
        if K > 0                                % validity check
            bs = K*(1 - p);
            if zeta < 1
                p1 = -2*exp(-zeta*wn*Ts)*cos(wn*Ts*sqrt(1 - zeta^2));
            else
                p1 = -2*exp(-zeta*wn*Ts)*cosh(wn*Ts*sqrt(zeta^2 - 1));
            end
            p2 = exp(-2*zeta*wn*Ts);
            KpS = min(max((p - p2)/bs, 0), Kp_max);           % limits
            KiS = min(max((1 + p1 + p2)/(bs*Ts), 0), Ki_max);
        end
    end
else
    KpS = Kp0;  KiS = Ki0;                      % adaptation switched off
end
%% 4. PI control law (velocity form, anti-windup by clamping)
e = r - double(y);
u = uPrev + (KpS + KiS*Ts)*e - KpS*ePrev;
u = min(max(u, 0), Vs);
uPrev = u;  ePrev = e;  Kp = KpS;  Ki = KiS;
end
```

Results: the response is shown in Figure 2. Before the change, the estimated gain settles at about 16 RPM/V and the gains at $K_p \approx 0.12$ V/RPM and $K_i \approx 0.54$ V/(RPM s). When the motor weakens at 40 s, the speed briefly falls below the reference. The estimated gain falls quickly, briefly undershooting while the estimate settles, so the controller first raises its gains sharply (to $K_p \approx 0.65$ V/RPM) and then lowers them again. Within about 12 s the estimated gain settles within 5% of 9.6 RPM/V, which is 60% of its original value, and the final gains are $K_p = 0.20$ V/RPM and $K_i = 0.92$ V/(RPM s), about 1.7 times their earlier values, as Equation (4) predicts. The RMS speed error was 3.48 RPM between 20 and 40 s and 3.63 RPM between 60 and 80 s, so the tracking performance after the change was almost the same as before it, because the controller restored its designed closed-loop poles.

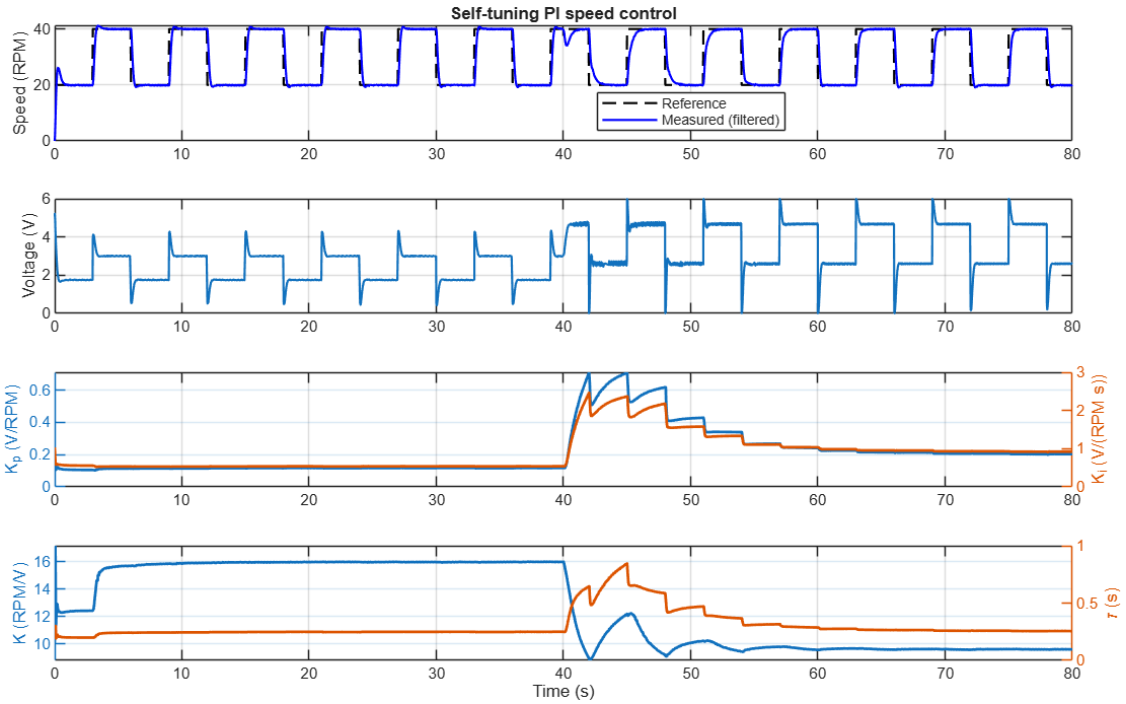


Figure 2: *Self-Tuning PI Control of the Virtual DC Motor: Speed (Top), Controller Output, PI Gains, and the Estimated Gain and Time Constant (Bottom); the Motor Gain Falls to 60% at t = 40 s*

Further Investigation

Investigate the behaviour of the self-tuning controller in `dc_motor_str_virtual.slx` by changing the parameters in the model workspace:

- **Forgetting factor:** repeat with $\lambda = 0.995$ and $\lambda = 0.98$. How long does each take to re-tune after the change, and how steady are the gains before it?
- **Load removed:** set `K_factor = 2.0`, so that the gain of the motor doubles at 40 s. Do the gains fall by the expected factor of two?
- **Design target:** repeat with $\omega_n = 1$ rad/s ($2\zeta\omega_n\tau = 0.5$). What happens to K_p, and why?
- **Excitation:** replace the square-wave reference with a constant 30 RPM from $t = 20$ s. What happens to the estimate and the gains after the load change, and why?

8.0 Summary

This Learning Guide introduced adaptive control, in which a controller adjusts its own parameters during operation. A change in the gain of a system is equivalent to scaling both PI gains, so a fixed controller cannot keep its designed closed-loop poles when the system changes. The self-tuning PI controller solves this by combining two earlier Learning Guides: recursive least squares estimates a first-order model of the motor every 0.05 s, and pole placement recalculates the PI gains from the latest model, using the certainty equivalence principle. Safeguards, including an initial guess, validity checks, gain limits, protection against covariance wind-up and a continuously changing reference, are essential for reliable operation on real hardware. The key progression introduced in this guide can be summarised as:

measure → estimate the model (RLS) → redesign the gains (pole placement) → control → repeat

In simulation, the self-tuning controller identified a 40% loss of motor gain within about 12 s and restored its designed response. The controller is put into practice on the ACE-Lab in the practical exercise, DC Motor Self-Tuning PI Control using Recursive Least Squares.

References

Åström, K. J. and Wittenmark, B. (1995). *Adaptive Control*, 2nd edn. Reading, MA: Addison-Wesley.

Ioannou, P. A. and Sun, J. (1996). *Robust Adaptive Control*. Upper Saddle River, NJ: Prentice Hall.

Recommended Citation

If you have used this learning guide in your teaching, learning or research, please cite it using the reference below, adapting the format as required by your preferred referencing style.

Pickering, J. E. (2026). Adaptive Control: the Self-Tuning PI Controller. ACE-Lab. Teaching Notes: ACE-Lab Learning Guide, Version 1.0. Zenodo. https://doi.org/10.5281/zenodo.23671545